DATA 441/641 · Applied Natural Language Processing · Week 15

15.2 Machine Translation

Map a sentence in one language to a sentence in another.

These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

1. The original NLP problem

Machine translation (MT) is automatic translation from one language to another. It is one of the oldest research problems in the field. The methods changed. The job did not: map a string in language L_1 to a string in L_2 that a reader of L_2 can use.

You will almost never train a production MT system from scratch in this course. You will decide **when to call one**, and how to live with its errors.

---

2. Three eras

Era	How it worked	What it needed
Rule-based	Grammars and dictionaries written by linguists	Expert time; brittle outside the grammar
Statistical	Phrase tables and language models from **parallel** text	Huge aligned corpora (Europarl and friends)
Neural	Encoder–decoder (and now transformers); the current default	Even more data and compute; Google Translate is the public example

Neural MT is state of the art in research and in the APIs you will call. Training one for a new pair is a large-organization project. That is a resource fact, not a value judgment.

---

3. Where industry uses MT without being "a translation company"

Two patterns from the lecture:

Normalize into one language, then run the rest of the stack. Reviews arrive in many languages. You want sentiment. Either you find a sentiment model per language, or you translate into one language and reuse the analyzer you already trust. The second path is often faster. It also inherits every MT mistake into the sentiment score.

Treat messy text as a translation problem. `am gud` → `I am good` is informal-to-standard English. If you have pairs, an MT-style model is a reasonable rewriter. This is not "translation" in the passport sense. It is the same machinery.

Other quiet uses: support tickets, product listings, and search queries that you want in the language of your index.

---

4. Practical advice

Call an API first. Building your own neural MT is rarely the bottleneck you think it is. Read the price sheet. Cache.

Keep a translation memory. Repeated strings (UI labels, boilerplate, frequent product names) should hit a store, not the API. Cheaper, more consistent, easier to correct by hand.

New language or a narrow domain. If the API is weak:

- a small rule layer for the phrases you must get right (drug names, legal formulae)
- **back-translation** to grow training data when you do train
- a **hybrid**: neural output plus rules and a post-edit filter when a wrong word is costly

Evaluation is its own field. BLEU and later metrics, human adequacy/fluency, dedicated conferences. If translation **is** the product, learn that literature. If translation is a preprocessor, measure the **downstream** task (sentiment F1, search NDCG), not only BLEU.

Names and codes. NER (Week 11) is useful here: lock person and product names so the MT system copies them. A translated drug name or a mangled SKU is worse than an awkward verb.

Do not chain blindly. Translate-then-classify is convenient. It also hides the language the user wrote. Keep the source text. When the downstream model is wrong, you need to know whether MT or the classifier failed.

---

5. What this note is for

You should be able to:

- place a system in the rule / statistical / neural timeline
- name two industry uses that are not "we are a translation company"
- choose API + memory over training, and say when a hybrid is worth it

---

6. The limit you should say out loud

Umberto Eco: "Translation is the art of failure." A model can be useful and still miss tone, idiom, and what the author left unsaid. Do not promise a perfect bilingual record. Promise a usable draft plus a place where a human can fix the lines that matter.

---

7. Practice

1. You have a strong English sentiment model and reviews in six languages. Sketch the pipeline, and name one way MT error becomes a sentiment error.
2. Why cache translations of a "Add to cart" button but not of a one-off tweet?
3. When would you add a rule on top of a neural API instead of fine-tuning?

Fine-tuning is for a domain you will stay in. A dozen locked strings are a rule file, not a training run.

If the client is a language-service company, this note is only the map. Their stack will be deeper than an API plus a cache.

For everyone else, the win is usually "good enough translation in the pipeline," not a new MT paper.